MAZE RACER

A First-Person 3D Maze Game in x86 Assembly

Computer Organization and Assembly Language (COAL) — Project Proposal

Submitted By:	Tayyaab Zahoor [*01-135232-070*]
	Anood Tayyeba [*01-135232-010*]
Program:	BS Information Technology — 6th Semester
Course:	Computer Organization & Assembly Language (COAL)
Submitted To:	Sir Adnan Jelani
Institution:	Bahria University
Date:	May 16, 2026

Contents

1. Introduction

This document is the project proposal for **Maze Racer**, a first-person 3D maze game built entirely in x86 assembly language using EMU8086. The project is being developed as part of the Computer Organization and Assembly Language (COAL) course in the 6th semester of BS Information Technology.

The idea behind this project is simple: instead of building something that just demonstrates a concept, we wanted to build something you can actually play. Maze Racer puts the player inside a procedurally generated maze where they have to find the exit before the timer runs out, all while being chased by enemies. Every mechanic is tied directly to assembly-level concepts we have covered in class.

The reason we chose a game over a utility program is that games make assembly concepts visible. When a wall gets darker as you walk away from it, that is a shift operation. When the enemy beeps faster as it gets closer, that is port I/O through the PC speaker. It turns something abstract into something you can see and hear in real time.

2. Project Overview

Maze Racer is a tile-based, first-person 3D maze game rendered using raycasting in VGA Mode 13h. The player navigates through a maze, finds the exit tile, and escapes before the countdown timer hits zero. An enemy patrols the maze and hunts the player down. If the enemy catches the player, the game ends.

The game has two difficulty modes:

- **Normal Mode** — The minimap shows the full maze layout and enemy positions. The timer is generous. Enemies are slow. This mode is meant for learning the controls and getting comfortable with the game.
- **Hard Mode** — The minimap only shows tiles the player has already visited. Enemies do not appear on the map at all. The only warning is a beeping sound through the PC speaker that gets faster and higher as an enemy gets closer. The timer is much tighter. This mode is genuinely scary and forces the player to think carefully about movement.

Each game session generates a new maze using a recursive backtracking algorithm, so no two runs are the same.

3. Objectives

The main objectives of this project are as follows:

1. Build a fully playable game from scratch using x86 assembly in EMU8086.
2. Implement a raycasting engine that renders a first-person 3D view using VGA Mode 13h.
3. Use `INT 1Ch` (timer interrupt) to handle the countdown clock and enemy movement independently of the keyboard.
4. Use `INT 16h` for real-time keyboard input without blocking the game loop.
5. Generate a unique maze every run using a recursive backtracking algorithm built on a stack in the `SS` segment.
6. Implement enemy AI using two different pathfinding strategies: wall-following and random movement.
7. Demonstrate PC speaker control via port `61h` for in-game audio feedback.
8. Store game state, map data, leaderboard scores, and flags in the `DS` data segment.
9. Provide two clearly different difficulty modes with distinct gameplay experiences.

4. Gameplay Description

4.1 Starting the Game

When the emulator boots, a title screen appears with the game name drawn in VGA pixel art. The player is prompted to choose between Normal and Hard mode. After selecting, a new maze is generated and the game begins.

4.2 Core Loop

The player starts at one corner of the maze. The exit tile is placed at the opposite corner. The player moves through the maze in first-person view, one tile at a time per keypress. The controls are:

Key	Action
`W`	Move forward
`S`	Move backward
`A`	Turn left
`D`	Turn right

Movement is tile-by-tile, not pixel-by-pixel. This keeps player position stored as a simple integer coordinate in registers (`BX` and `SI`), which eliminates any floating-point complications.

The minimap is displayed in a corner of the screen. In normal mode it shows everything. In hard mode it only reveals tiles the player has stepped on.

4.3 Enemies

Enemies move one tile per timer tick. Their speed is controlled by a single value, which makes difficulty tuning easy. In normal mode, enemies appear as markers on the minimap. In hard mode, they are completely invisible on the map. The only way to detect them is through the PC speaker, which beeps faster and at a higher pitch the closer an enemy is.

Two enemy types exist in hard mode:

- One enemy uses a wall-following strategy, always turning left when possible.
- The other enemy moves randomly. This means the two enemies behave very differently, which creates the feeling of intelligent hunting even though the logic behind each is minimal.

4.4 Winning and Losing

If the player reaches the exit tile before the timer runs out, the screen flashes gold. A score breakdown appears showing time bonus, level bonus, and a difficulty multiplier. Best times for each difficulty are stored in a memory array in `DS` and shown on the title screen.

If the timer hits zero or an enemy catches the player, a death animation plays. The screen floods with red columns starting from the direction the enemy came from. This takes about two seconds and is handled entirely by a VGA column-fill loop.

5. Technical Design

5.1 VGA Rendering — Mode 13h

The 3D view is drawn using a raycasting technique in VGA Mode 13h, which gives us a 320x200 pixel framebuffer accessible through segment `0A000h`. For each column on the screen, a ray is cast from the player's position in the direction they are facing. The distance to the nearest wall is calculated. Walls that are farther away are drawn shorter and darker using the `SHR` (shift right) instruction on the brightness value.

A torch flicker effect is added by applying a small `XOR` to the brightness value every few frames. This costs almost nothing in terms of code but gives the visuals a lot of atmosphere.

5.2 Maze Generation — Recursive Backtracker

Each new game generates a fresh maze using the recursive backtracking algorithm. This algorithm works by starting at a random cell, carving passages to unvisited neighbors, and backtracking when it hits a dead end. In assembly, the recursion is simulated using a manual stack in the `SS` segment. The result is a perfect maze, meaning there is exactly one path

between any two cells, which keeps navigation fair.

5.3 Timer and Interrupts

The countdown timer is driven by `INT 1Ch`, which fires roughly 18.2 times per second. Each time the interrupt fires, a counter in `DS` is decremented. When it hits zero, the game ends. Enemy movement is also tied to this interrupt, so enemies move at a fixed real-world speed regardless of anything else happening in the game loop.

Keyboard input is handled through `INT 16h` in non-blocking mode, meaning the game does not freeze while waiting for a key.

5.4 PC Speaker Audio

The PC speaker is controlled by writing directly to port `61h` via the `OUT` instruction. When an enemy is nearby, the beep frequency increases and the beeping gets faster. In hard mode, this is the player's only warning. It is also a clean demonstration of hardware-level port I/O directly from lecture material.

5.5 Data Storage

All game data lives in the `DS` segment:

- Maze grid stored as a byte array (each byte is a cell; bits represent which walls are open)
- Visited tile flags stored as a bit array for the hard mode minimap
- Enemy positions and state flags
- Timer value and score
- Leaderboard array storing best times for normal and hard mode

6. Features at a Glance

Feature	Description	COAL Concept
3D Renderer	First-person raycasting view in VGA Mode 13h	`MUL`, `SHR`, `ES:0A000h`
Maze Generation	Recursive backtracker using a manual stack	Stack in `SS`, conditional jumps
Tile-based Movement	Player moves one tile per keypress	Register arithmetic, `CMP+JZ`
Timer Countdown	Real-time clock using interrupt	`INT 1Ch`
Keyboard Input	Non-blocking key detection	`INT 16h`
Enemy Pathfinding	Wall-following and random movement	Memory lookup, conditional logic
PC Speaker Beeping	Proximity alert via port I/O	`OUT 61h`
Normal Mode Minimap	Full map visible including enemies	Bit array in `DS`
Hard Mode Minimap	Only visited tiles shown, no enemies	Bitwise flags per tile
Torch Flicker	Subtle wall brightness variation	`XOR` on shading value
Death Animation	Screen floods red from enemy direction	VGA column loop
Victory Screen	Score breakdown with time and difficulty bonus	`MUL` for score calculation
Leaderboard	Best times stored across sessions	Array addressing in `DS`
Title Screen	Pixel art intro before the game starts	VGA pixel art, `INT 10h`
Two Difficulty Modes	Normal and Hard with distinct gameplay	Bitwise mode flags

7. Team Responsibilities

The work is split between two members based on the natural boundaries of the project.

7.1 Anood — Rendering and World

- VGA Mode 13h setup and framebuffer access

- Raycasting draw loop and wall height calculation
- Wall shading by distance using `SHR`
- Ceiling and floor color rendering
- Torch flicker effect via `XOR`
- Maze generation using recursive backtracker
- Level data and map storage in `DS`
- Exit tile placement and detection

7.2 Tayyaab — Game Logic and Systems

- Keyboard input via `INT 16h`
- Player movement, rotation, and collision detection
- Timer countdown using `INT 1Ch`
- Enemy movement and pathfinding (wall-follower and random)
- PC speaker proximity beeping via `OUT 61h`
- Minimap rendering (full mode and visited-only mode)
- Score calculation and leaderboard storage
- Death animation and victory screen
- Title screen and difficulty selection

8. Development Timeline

Week	Phase	Goals
1	Setup and Planning	EMU8086 environment, VGA mode test, project structure
2	Core Renderer	Raycasting engine, basic wall drawing in Mode 13h
3	World and Movement	Maze generation, player movement, collision detection
4	Game Systems	Timer interrupt, keyboard input, enemy movement
5	Audio and Minimap	PC speaker beeping, minimap with visited tile logic
6	Difficulty Modes	Normal and Hard mode toggle, enemy map visibility flags
7	Polish	Death animation, victory screen, title screen, leaderboard
8	Testing and Presentation	Full playtesting, bug fixes, demo preparation

9. Tools and Environment

- **Assembler and Emulator:** EMU8086
- **Language:** x86 Assembly (8086 instruction set)
- **Graphics Mode:** VGA Mode 13h (320x200, 256 colors)
- **Audio:** PC Speaker via port `61h`
- **Version Control and Collaboration:** Overleaf (report), shared project files
- **Documentation:** Latex on Overleaf

10. Why This Project

Most assembly projects are demonstrations of a concept, and once you have seen the concept once, there is not much more to look at. A game is different. Every part of the program has a reason to exist, and every concept from class maps to something the player can actually see or feel.

The raycasting renderer shows bitwise operations and memory-mapped I/O. The maze generator shows stack usage and recursive logic. The timer and keyboard systems show interrupt handling. The PC speaker shows hardware port control. And it all comes together into something that is actually fun to play.

We chose not to include combat or health points. Keeping it as a pure evasion game means one touch from an enemy ends the run. This is actually better game design because it keeps

tension high and also keeps the code simpler. There is no need for hit detection, attack logic, or health bar rendering.

The goal is to walk into the presentation, boot the emulator, and show something that no one else in the room has built before.

11. Conclusion

Maze Racer is a first-person 3D maze game written entirely in x86 assembly. It covers timer interrupts, VGA rendering, stack-based algorithms, port I/O, interrupt-driven input, and data storage in the `DS` segment. It has two difficulty modes, procedurally generated mazes, enemy pathfinding, a proximity audio system, a leaderboard, and a full presentation-ready title screen.

Every feature has a direct connection to something from the COAL curriculum. The project is designed to be buildable by a two-person team over eight weeks without cutting corners on quality.

> *"The goal of this project is not just to write assembly code that compiles. It is to write assembly code that someone actually wants to play."*

Maze Racer — COAL Project Proposal
BS Information Technology, 6th Semester
May 16, 2026